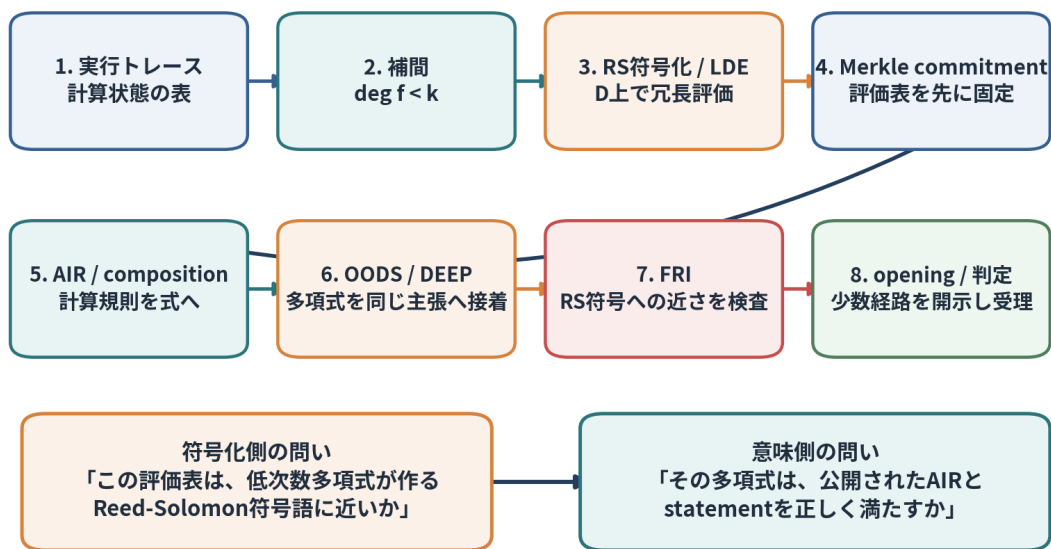


ZK-STARK

Reed-Solomon 符号化と検証プロセス

「なぜ符号化するのか」と「検証者は何をどの順序で確かめるのか」を結び付ける詳細図解

Reed-Solomon符号化とSTARK検証の因果連鎖



両方の検査が成立して初めて、検証者は計算証明を受理できる。

図 0 Reed-Solomon 符号化と STARK 検証の因果連鎖（筆者作成）

2026 年 8 月 補強資料

本資料の目的と読み方

本資料は、前資料で扱った LDE、AIR、制約の束ね、OODS/DEEP、FRI をさらに補強し、ZK-STARK における Reed-Solomon 符号化と、検証者が証明を受理するまでの検査手順を一つの因果関係として説明するものである。特に、「Reed-Solomon 符号は誤り訂正符号なのに、なぜ証明で使えるのか」「FRI は何を証明し、何を証明しないのか」「検証者は計算全体を再実行せず、なぜ少数の値だけで判断できるのか」という疑問へ答える。

最重要の見取り図 Merkle commitment は値を固定し、AIR は計算の意味を定め、OODS/DEEP は複数の多項式を同じ主張へ結び付け、FRI は最終的な評価表が低次数 Reed-Solomon 符号へ近いことを検査する。どれか一つだけでは、ZK-STARK の健全性を説明できない。

対象読者と前提

ZKP 共通の完全性・健全性・ゼロ知識性の一般論は省略する。有限体、多項式、Merkle 木、AIR については、本文の理解に必要な範囲で再説明する。本文は古典的な乗法部分群上の Reed-Solomon 符号と二分 FRI を中心に説明する。Circle STARK / Circle FRI などの現代的実装では評価領域と folding の代数が異なるが、「低次数評価符号へ近いことを commitment とランダム照会で確認する」という役割は共通である。

章構成

章	内容
第 1 章	Reed-Solomon 符号が STARK のどこに位置するか
第 2 章	トレースから RS 符号語を作る手順と式
第 3 章	距離、冗長性、1 箇所の嘘の正確な理解
第 4 章	RS membership と proximity、FRI の役割
第 5 章	検証者が受け取る proof object と固定すべき設定
第 6 章	検証者の一連の処理を 10 ステップで説明
第 7 章	攻撃と検査の対応、パラメータと実装上の注意
Appendix A	F ₁₇ 上の RS 符号化と最小距離の数値例
Appendix B	F ₁₇ 上の FRI query 検証の数値例

第1章 Reed-Solomon 符号が ZK-STARK のどこに位置するか

ZK-STARK では、証明者が作成した実行トレースを、そのままランダムに抜き取って検査するわけではない。通常の表では各セルが独立しており、検査されない場所へ不正を隠せるからである。そこで、各トレース列を低次数多項式へ補間し、より大きな評価領域で評価する。この評価ベクトルが Reed-Solomon 符号語である。

同じReed-Solomon符号でも、目的が異なる

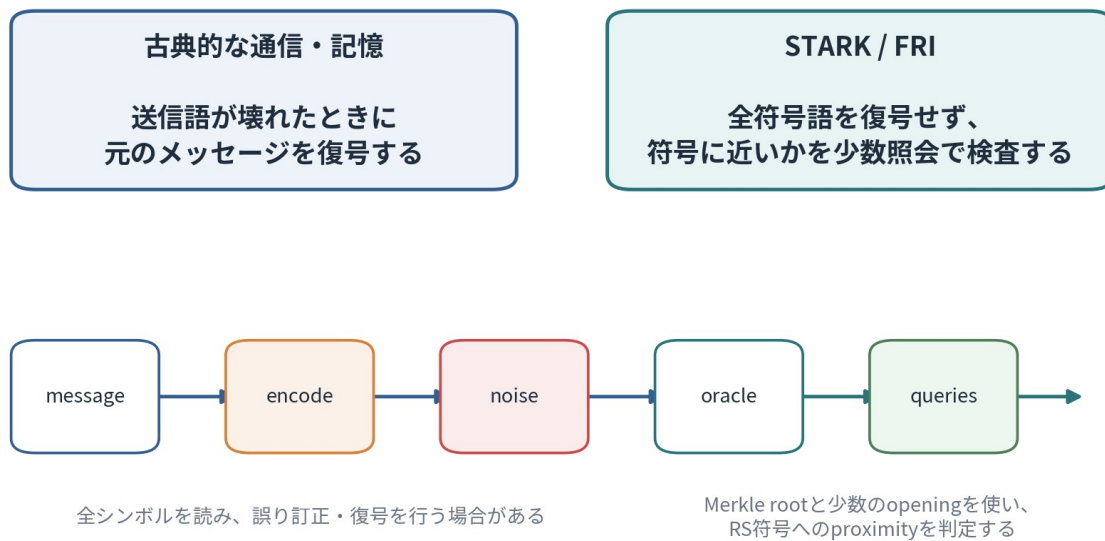


図1 古典的な誤り訂正と STARK における proximity test の違い (筆者作成)

1.1 古典的用途は復号、STARK での用途は構造検査である

Reed-Solomon 符号は、通信路や記憶媒体で壊れたシンボルから元のメッセージを復元する誤り訂正符号として知られている。一方、STARK の検証者は通常、評価表全体を受信して復号するわけではない。証明者が評価表へ Merkle commitment を行い、検証者は少数の位置だけを開かせる。FRI は、この oracle が低次数多項式の評価、すなわち Reed-Solomon 符号語に近いかを検査する IOP of Proximity である。[2][3]

$$RS_D(k) = \{ (f(d_0), f(d_1), \dots, f(d_{n-1})) : \deg f < k \}$$

$D=\{d_0, \dots, d_{n-1}\}$ は互いに異なる評価点の集合である。

注1 「符号語に近い」とは、評価ベクトルのうち、少数の成分を書き換えればある正規 RS 符号語になることを意味する。FRI は一般に exact membership ではなく proximity を検査する。

1.2 Reed-Solomon 符号だけでは計算の正しさを証明しない

低次数多項式は計算内容を知らない。証明者が誤った実行トレースを作り、そのトレースを一貫して低次数多項式へ補間すれば、結果は正規の Reed-Solomon 符号語になり得る。この場合、FRI だけを実行しても低次数性は確認できるため、誤った計算を排除できない。計算の意味は AIR と境界制約が定め、その AIR 制約から作られた composition polynomial を OODS/DEEP で低次数検査へ結び付ける必要がある。

STARKの四つの検査層: どの攻撃を誰が止めるか

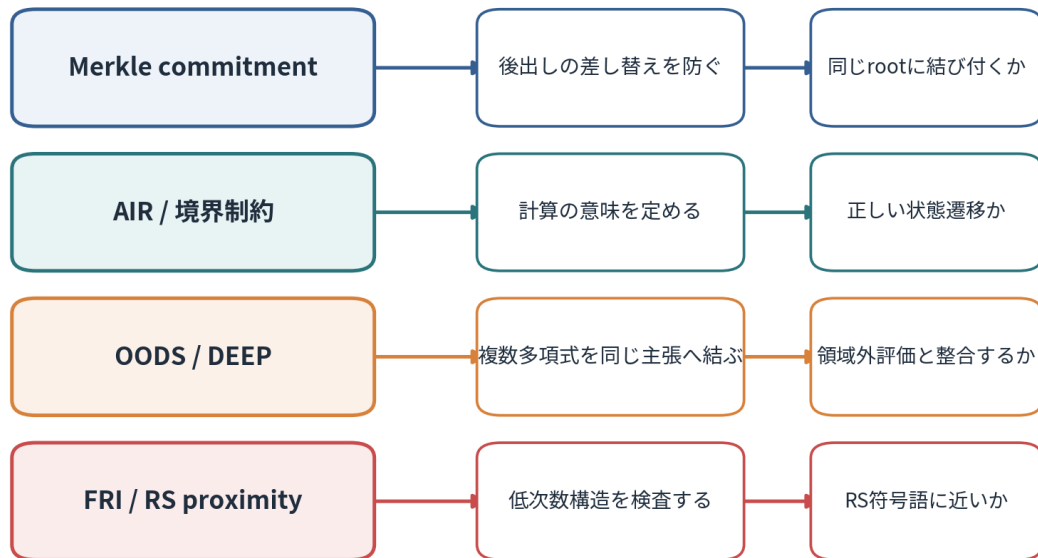


図2 Merkle、AIR、OODS/DEEP、FRIの役割分担（筆者作成）

誤解しやすい点 「LDEによって1箇所の嘘が広がるから、FRIだけで不正計算を検出できる」という説明は不十分である。誤ったトレースを正しく再符号化した場合、その結果も valid codeword であり、FRIは低次数性については正しく受理し得る。AIRとの結び付きが必須である。

第2章 トレースから Reed-Solomon 符号語を作る

2.1 三段階: トレース値、補間多項式、拡張評価

トレース列を $a=(a_0, \dots, a_{k-1})$ とし、互いに異なるトレース点 $H=\{h_0, \dots, h_{k-1}\}$ を選ぶ。証明者は、 $f(h_i)=a_i$ を満たす次数 k 未満の多項式 f を補間する。次に、より大きい評価領域 $D=\{d_0, \dots, d_{n-1}\}$ において同じ f を評価し、長さ n のベクトルを作る。このベクトルが $RS_D(k)$ の符号語であり、STARK 文脈では LDE 評価表に相当する。

STARKにおけるReed-Solomon符号化: H上の値からD上の符号語へ

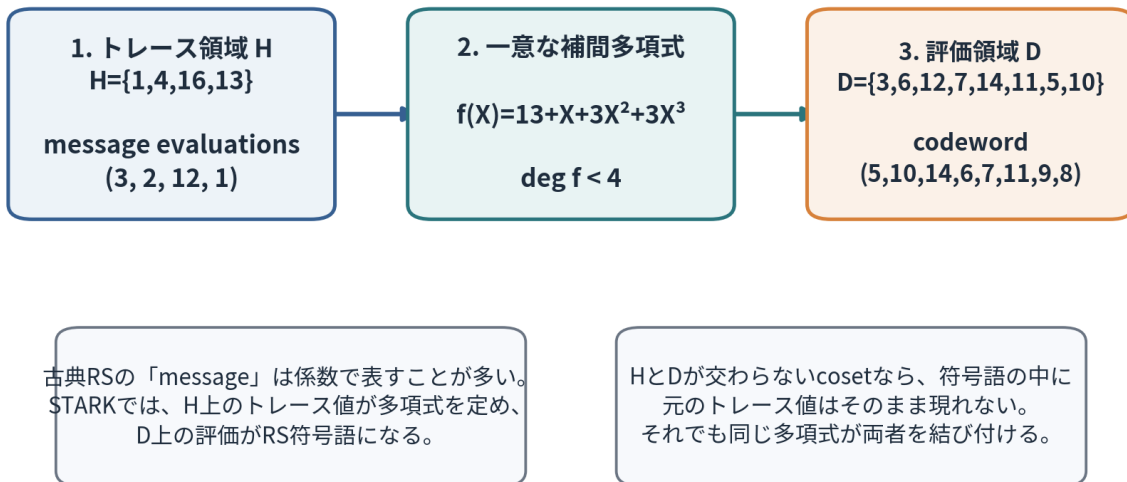


図3 F₁₇上のトレース値から Reed-Solomon 符号語を作る三段階（筆者作成）

$$\text{Enc}_D(a) = (f(d_0), \dots, f(d_{n-1})), \text{ where } f(h_i)=a_i \text{ and } \deg f < k$$

注2 古典的な RS 符号では、メッセージを多項式係数として扱う定義が多い。STARK では、トレース領域 H 上の評価値が多項式を定める評価形式を用いる。両者は線形変換で対応する。

2.2 systematic である必要はない

H が D の部分集合であれば、符号語の中に元のトレース値がそのまま含まれるため systematic に近い見方ができる。しかし、実用 STARK では、分母のゼロを避けることや FFT 構造を利用することなどから、トレース領域と LDE 領域に異なる coset を用いる場合がある。この場合、 D 上の符号語は元のトレース値を直接含まないが、同じ補間多項式 f によって厳密に結び付いている。

2.3 $[n, k, d_{\min}]$ 、符号率、blowup factor

RSコードの三つの基本量: $[n, k, d_{\min}]$



図4 RSコードの長さ、自由度、最小距離と blowup factor (筆者作成)

$$\text{rate } \rho = k/n, \quad \text{blowup } b \approx n/k = 1/\rho, \quad d_{\min} = n-k+1$$

n を大きくし、 k を同じままにすると符号率は下がり、冗長な評価点が増える。符号語同士はより大きな相対距離を持ちやすくなるため、proximity test の健全性に有利である。一方、証明者はより多くの LDE 値を計算して Merkle 木へ格納する必要があり、証明生成時間とメモリが増える。したがって blowup factor は、安全性と証明者コストを結ぶ主要パラメータである。

2.4 FFT が使われる理由

評価領域を乗法部分群またはその coset として選ぶと、補間と多点評価を FFT 型アルゴリズムで効率化できる。単純な方法では k 個の係数を n 点へ評価する処理に大きなコストがかかるが、構造化された領域では概ね準線形時間で LDE を計算できる。STARK の証明者が大規模トレースを扱える理由の一つは、この評価領域の代数構造にある。[1][2]

第3章 距離と冗長性: 「1箇所の嘘」を正確に理解する

3.1 最小距離は多項式の根の個数から生じる

二つの異なる次数 k 未満の多項式 f と g を考える。対応する符号語がある評価点 d_i で一致するなら、 $h=f-g$ は d_i を根に持つ。 h は次数 k 未満の非零多項式であるため、異なる根を高々 $k-1$ 個しか持てない。したがって二つの符号語は高々 $k-1$ 位置でしか一致せず、少なくとも $n-k+1$ 位置で異なる。

なぜ $d_{\min} = n-k+1$ なのか: 根の個数で説明する

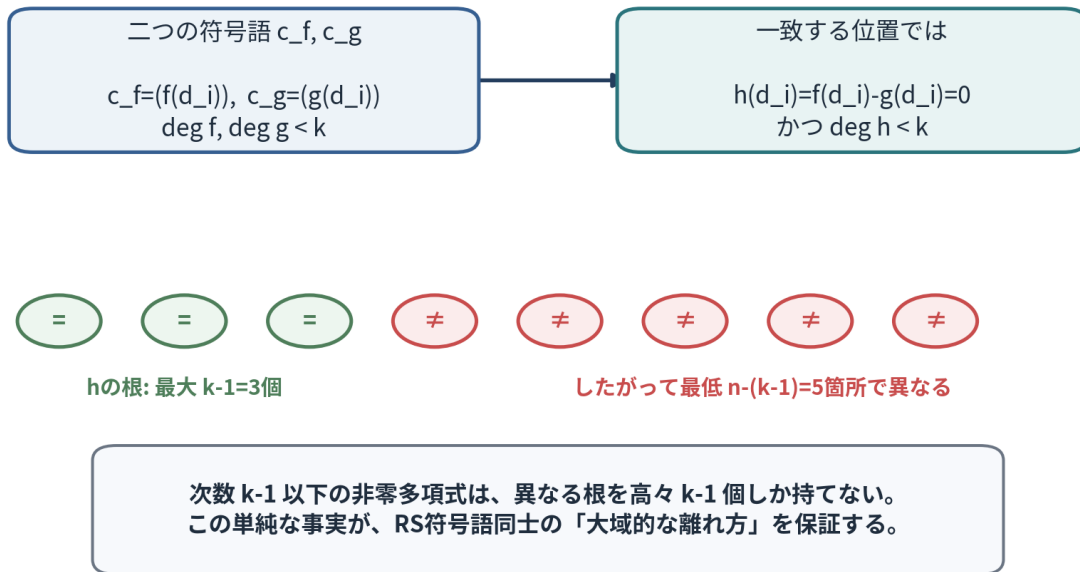


図5 RS符号の最小距離を根の個数から導く (筆者作成)

$$\text{dist}(c_f, c_g) \geq n-(k-1) = n-k+1$$

3.2 トレースの1セルを変えて再符号化すると何が起こるか

トレース領域 H 上の第 i 成分だけを Δ 変更すると、新しい補間多項式は $f'(X)=f(X)+\Delta L_i(X)$ となる。 L_i は H 上では第 i 点だけで1、他の点で0となる Lagrange 基底である。 D が H と交わらない適切な領域であれば、 $L_i(d)$ は多くの $d \in D$ で非零となるため、変更の影響が多数の LDE 評価へ現れる。

$$f'(X) = f(X) + \Delta L_i(X), \quad L_i(h_i)=1, \quad L_i(h_j)=0 \ (j \neq i)$$

しかし、 f' 自体は依然として次数 k 未満の正規多項式である。したがって、 $f'|_D$ は別の valid RS codeword である。これは「変更が多数点へ波及する」という説明と矛盾しない。波及は符号語同士を遠ざけるが、どちらの符号語も低次数であるため、FRI だけではどちらが正しい計算を表すかを判断できない。

3.3 LDE 後の 1 シンボルだけを改ざんした場合

一方、正しい符号語 c のうち一成分だけを書き換えたベクトル w は、一般には RS 符号語ではない。ただし w は c から距離 1 にあり、符号集合には近い。proximity test は「十分遠い語を高確率で拒否する」ことを目的とするため、1 シンボルだけの誤りを必ず拒否するとは限らない。単純なランダム照会だけなら、その位置を引く確率は q/n 程度である。実際の FRI 健全性は folding、リスト復号半径、field 項などを含むため、次式は直観にすぎない。

$$\Pr[\text{all } q \text{ direct queries miss } \delta n \text{ bad positions}] \approx (1-\delta)^q$$

FRI の厳密な soundness bound ではない。距離と照会回数の感覚を得るための近似である。

「1箇所の嘘」を二種類に分けることが重要である

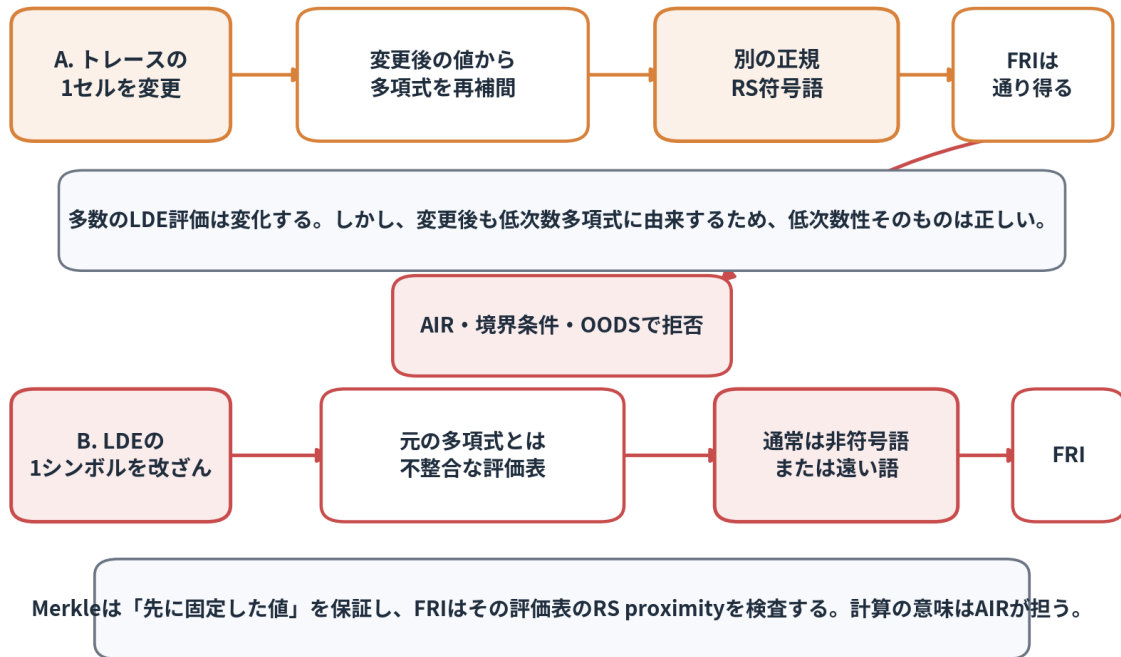


図6 トレースの嘘と LDE シンボルの嘘を分けて考える (筆者作成)

正しい結論 Reed-Solomon 符号の大域的距離は、不正な多項式候補を広く分離する。しかし、計算の正しさを決めるのは AIR であり、FRI はその AIR 由来の多項式が低次数構造を持つことを検査する。

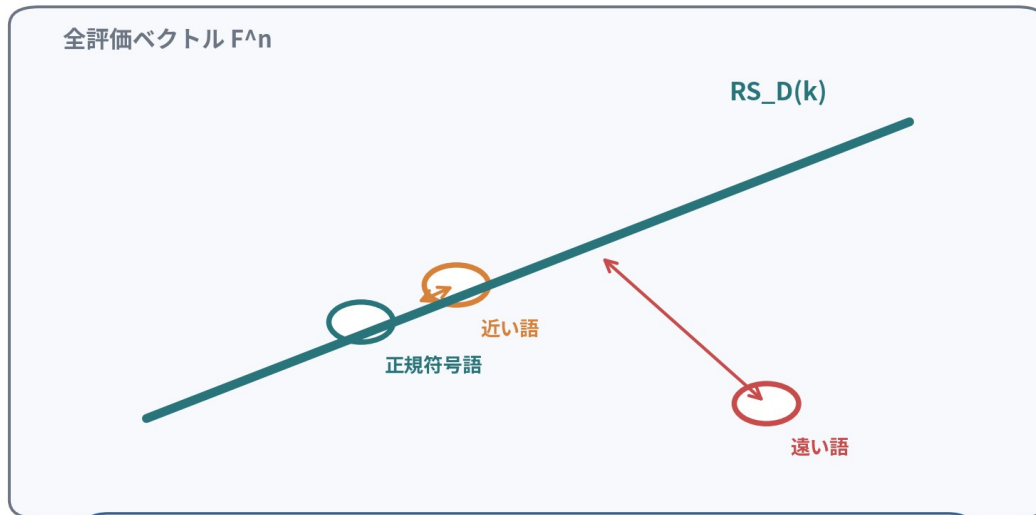
第4章 RS membership、proximity、FRI

4.1 正規符号語、近い語、遠い語

評価ベクトル全体 F^n の中で、次数 k 未満の多項式から得られるベクトルだけが $RS_D(k)$ に属する。任意のベクトル w について、RS 符号への相対 Hamming 距離を、最も近い符号語までの不一致割合として定義する。

$$\Delta(w, RS_D(k)) = \min_{\{c \in RS_D(k)\}} |\{i : w_i \neq c_i\}| / n$$

RS符号語集合と proximity の直観



FRIは「正規符号語そのもの」への完全一致ではなく、通常は距離 $\text{dist}(w, RS_D(k))$ が十分小さいかを検査するIOPPである。

図7 RS符号語集合と proximity の幾何学的直観（筆者作成）

4.2 FRI が証明するもの

FRI は、証明者が commit した oracle が所定の Reed-Solomon 符号へ近いことを示す IOP of Proximity である。証明者は評価関数を偶数・奇数成分へ分解し、検証者のランダム challenge によって次の低次数候補へ fold する。各ラウンドで次数上限と評価領域が縮小し、最後は小さな多項式を直接検査できる。[3]

$$f(X)=f_{\text{even}}(X^2)+X f_{\text{odd}}(X^2), \quad f_{\text{next}}(Y)=f_{\text{even}}(Y)+\beta f_{\text{odd}}(Y)$$

$$f_{\text{next}}(x^2) = (f(x)+f(-x))/2 + \beta \cdot (f(x)-f(-x))/(2x)$$

4.3 FRI が証明しないもの

FRI は、低次数多項式の意味、境界値、プログラムの命令、署名検証、残高保存などを理解しない。また、Merkle root が正しいプログラムの trace を commit していることも単独では保証しない。FRI に渡す多項式が AIR 制約や公開 statement へ正しく結び付いていることを、composition polynomial、OODS、DEEP、および query opening の整合性で確認する必要がある。

役割の境界 FRI の質問は「この oracle は低次数 RS 符号に近い」である。STARK 全体の質問は「公開 statement を満たす計算から作られた oracle か」である。後者は AIR・commitment・linking・FRI の合成によって初めて成立する。

4.4 DEEP-FRI が領域外を使う理由

list decoding 領域では、一つの評価ベクトルの近くに複数の低次数多項式候補、いわゆる pretenders が存在し得る。DEEP-FRI は評価領域 D の外のランダム点 z で多項式値を要求することで、証明者に一つの候補を選ばせ、soundness を改善する。[4]

$$q_z(X) = (f(X) - f(z)) / (X - z)$$

第5章 検証者が受け取るものと、事前に固定すべきもの

5.1 検証入力は proof だけではない

検証者は、proof bytes だけを見て検証してはならない。公開 statement、program または AIR の識別子、有限体、評価領域、次数上限、hash 関数、query 数、blowup factor、FRI 設定、公開入力の serialization 規則などを、検証者側の信頼された設定として持つ必要がある。

区分	代表的な内容	誰が固定するか
公開 statement	public inputs、output、nonce、state root	アプリケーション / verifier
計算識別	program hash、AIR ID、回路・VM バージョン	verifier / governance
代数設定	field、domain、degree bounds、extension field	verifier 実装または承認 manifest
安全性設定	blowup、query 数、FRI arity、grinding	verifier policy
proof data	commitments、OODS 値、FRI roots、openings	prover が提示し、verifier が検査

セキュリティ上の必須事項 証明者が proof 内で指定した query 数、blowup factor、field extensionなどを、検証者がそのまま採用してはならない。これらは soundness を直接左右するため、検証者側で固定または厳格に許可リスト化する必要がある。過去には、この境界を誤った STARK 実装に対する重大な指摘が公開されている。[9]

5.2 proof object の概念構造

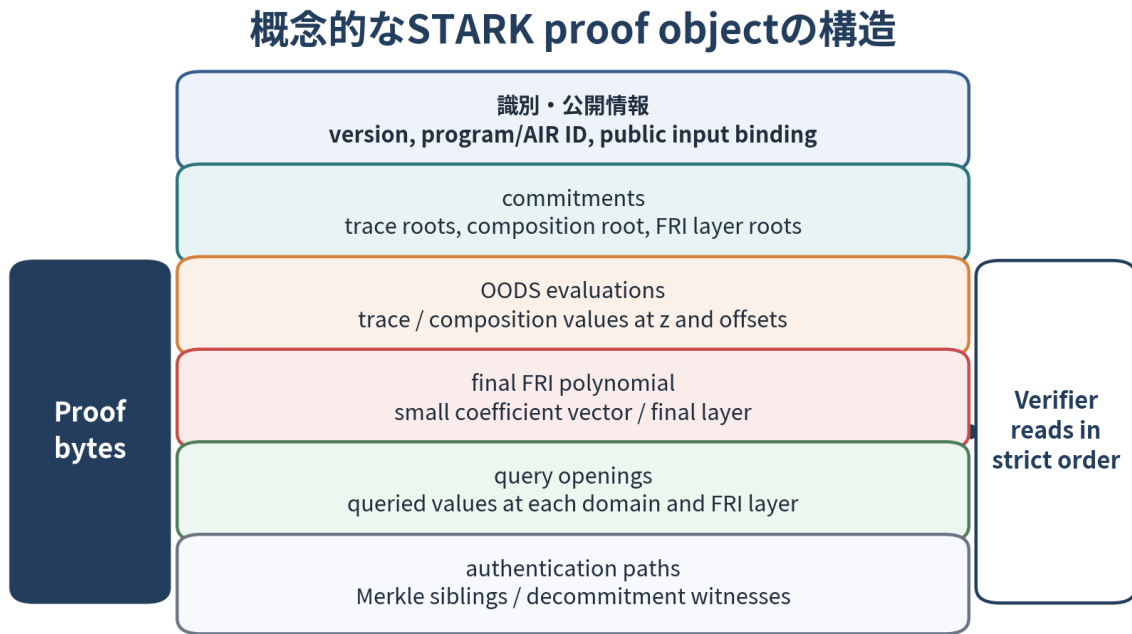


図 8 概念的な STARK proof object の構成要素（筆者作成）

実際の証明フォーマットは実装によって異なる。例えば複数列を一つの Merkle 木へまとめる実装、異なる log size の列を同じ commitment scheme で扱う実装、Circle FRI を用いる実装、proof-of-work grinding を含む実装などがある。重要なのは、各値が Fiat-Shamir transcript のどの時点で吸収され、どの challenge に先行するかを仕様で固定することである。

5.3 検証者は何をしないか

検証者が「すること」と「しないこと」

すること

- 公開statementとprogram IDを固定
- commitmentとopeningを照合
- AIR/OODS/DEEPの式を再計算
- FRIの少数経路と最終次数を確認

しないこと

- 実行トレース全体を受け取る
- プログラムを最初から再実行する
- RS符号語全体を復号する
- witnessを知る

検証の効率は、「全データを読む代わりに、
低次数構造・commitment・ランダム照会を組み合わせる」ことから生まれる。

図9 検証者が実行する処理と実行しない処理（筆者作成）

検証者は、witness や実行トレース全体を受け取らず、プログラムを最初から再実行しない。RS 符号語全体を復号することもない。代わりに、公開 statement に結び付いた少数の evaluation opening、Merkle path、OODS 関係式、FRI folding を確認する。この非対称性が、長い計算に対する短時間検証を可能にする。

第 6 章 ZK-STARK 検証プロセス: 最初から最後まで

検証者の10ステップ: 検証は「全計算の再実行」ではない



図 10 検証者が実施する 10 段階の全体フロー（筆者作成）

ステップ 1 検証者は、statement・program ID・安全性設定を固定する

検証者は、証明が何を主張するものかを最初に固定する。公開入力だけでなく、どの AIR またはプログラムを受理するか、どの有限体・評価領域・次数上限・hash 関数を使うかを確定する。これをしないと、攻撃者が「常に true を返す別プログラム」や極端に弱い安全性設定の proof を提出できる。目的は、証明対象と検証ルールを攻撃者から独立させることである。

ステップ 2 検証者は、proof の構造・長さ・値域を検査する

検証者は、commitment の個数、FRI 層数、opening 数、final polynomial の係数数、field element の正規表現、index の範囲などを確認する。malformed proof を後段の処理へ渡すと、index 誤り、DoS、曖昧な serialization などにつながる。目的は、以後の数値検査が仕様どおりのオブジェクトへ適用される状態を作ることである。

ステップ 3 検証者は、Fiat-Shamir transcript を再構築する

検証者は、public statement、program ID、各 Merkle root、OODS 値、FRI rootなどを、仕様で定めた順序と domain separation tag で channel へ吸収する。証明者が生成時に得たものと同じランダム係数、OODS 点、FRI folding 係数、query index を再現する。目的は、証明者が challenge を選ばないようにし、全 proof を一つの改変不能な因果連鎖へすることである。

Fiat-Shamirの依存関係: commitmentの後にchallengeが決まる

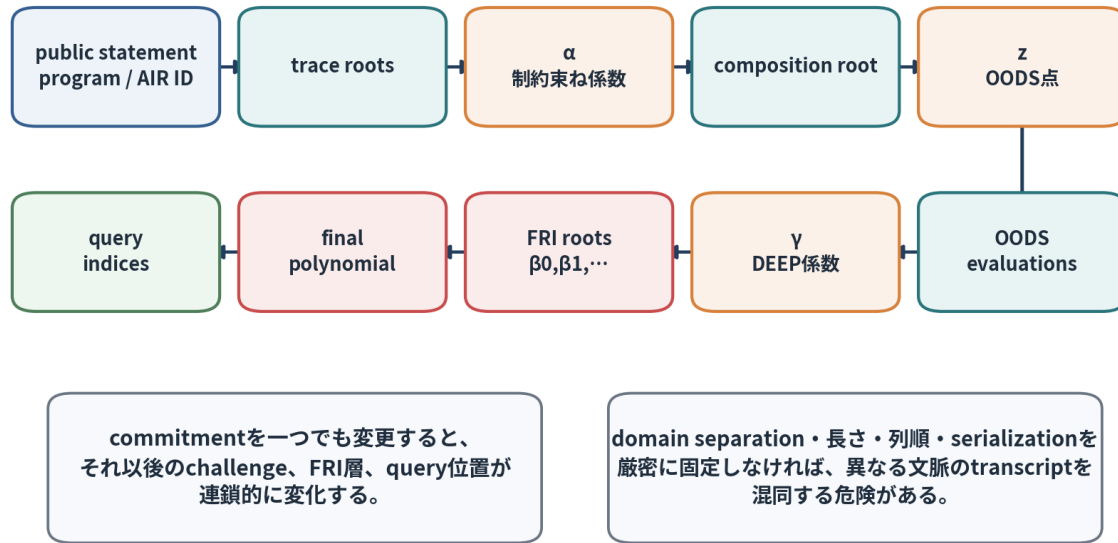


図 11 commitment と challenge の Fiat-Shamir 依存関係 (筆者作成)

ステップ4 検証者は、commitment root と次数上限を登録する

検証者は、trace、interaction trace、composition polynomial、FRI 各層などの root を commitment scheme へ登録する。このとき、各 commitment が表す列数・log size・degree bound を検証者側の設定と対応させる。root は「何らかのデータを固定した」ことしか示さないため、後続の opening と低次数検査が必要である。

ステップ5 検証者は、OODS 点と制約束ね係数を再生成する

trace commitment 後に制約のランダム合成係数 α を生成し、composition commitment 後に評価領域外の点 z を生成する。 z は通常の LDE 評価表に含まれないため、証明者が事前に表の値を調整するだけでは対応しにくい。目的は、複数の多項式候補の曖昧さを減らし、trace と composition を領域外の一点で結び付けることである。

ステップ6 検証者は、OODS 点で AIR と composition の等式を検査する

証明者が提示した trace 列の z および必要な offset 点での評価から、検証者は AIR 制約の分子 $N_j(z)$ 、消失多項式 $Z_j(z)$ 、ランダム係数 α_j を使って composition polynomial の期待値を再計算する。提示された $Q(z)$ と一致しなければ拒否する。Stwo の公式検証フローでも、OODS で再構成した composition 評価と proof 内の値の一致が独立した sanity check として実施される。[6]

$$Q(z) \stackrel{?}{=} \sum_j \alpha_j \cdot N_j(z) / Z_j(z)$$

OODS / DEEPの検証: 「値の束」と「低次数証明」を接着する

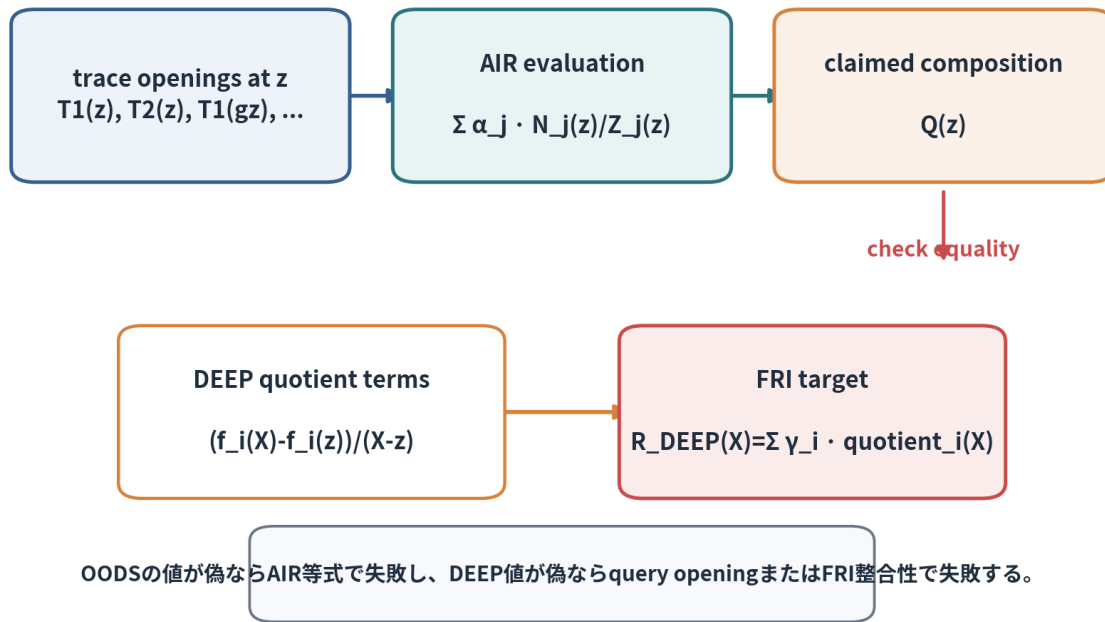


図 12 OODS での AIR 等式と DEEP を介した FRI への接続 (筆者作成)

ステップ 7 検証者は、すべての FRI commitment 後に query 位置を生成する

FRI の各ラウンドでは、証明者が現在層の root を commit した後に folding challenge β_r が生成される。最後の root と final polynomial まで transcript へ吸収した後で、実際に開く query 位置を生成する。目的は、証明者が検査される位置を知ってから評価表や FRI 層を作り直すことを防ぐことである。

ステップ 8 検証者は、各 opening の Merkle path を検査する

検証者は、query index、開示値、兄弟 hash を使って leaf から root までを再計算する。左右の順序、index の bit、複数列の leaf encoding、domain separation を仕様どおりに扱い、再計算 root が commitment と一致することを確認する。目的は、後続の数式で使う値が、証明者が先に固定した oracle の一部であると保証することである。

Merkle openingの検証: 1個の値をrootへ戻す

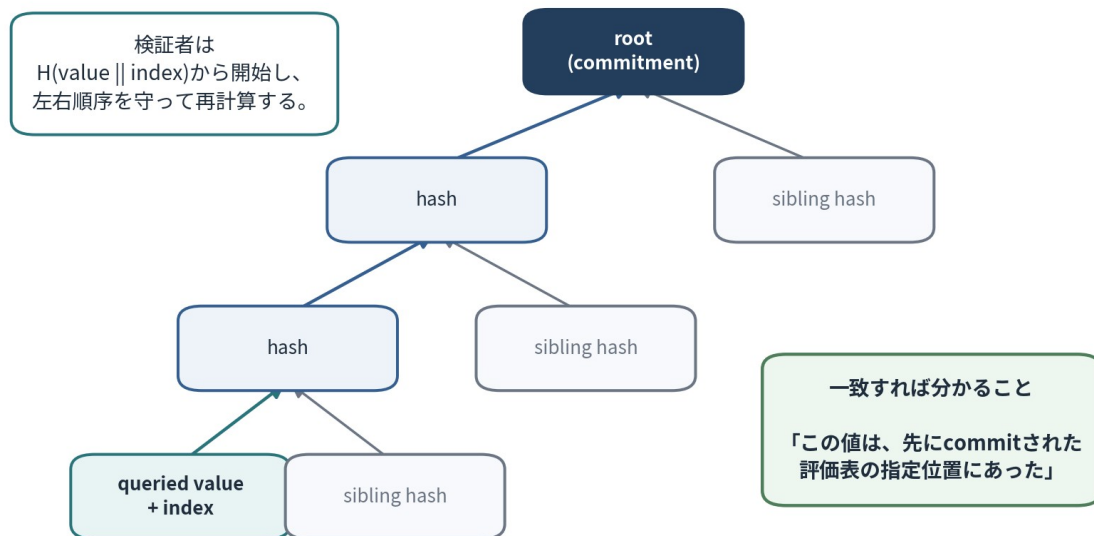


図 13 Merkle authentication path の再計算 (筆者作成)

ステップ9 検証者は、DEEP 値と FRI folding を層ごとに検査する

検証者は、開示された trace・composition 値と OODS 評価から、その query 位置における DEEP composition 値を再構成する。次に、各 FRI 層で x と対となる点の値を使い、fold 式から次層の期待値を計算し、開示された次層の値と一致するかを確認する。この処理を最終層まで繰り返す。目的は、各 FRI root が互いに無関係な評価表ではなく、一つ前の層をランダムに fold して得られた系列であると保証することである。

FRI query path: 同じランダム経路を各層で追跡する

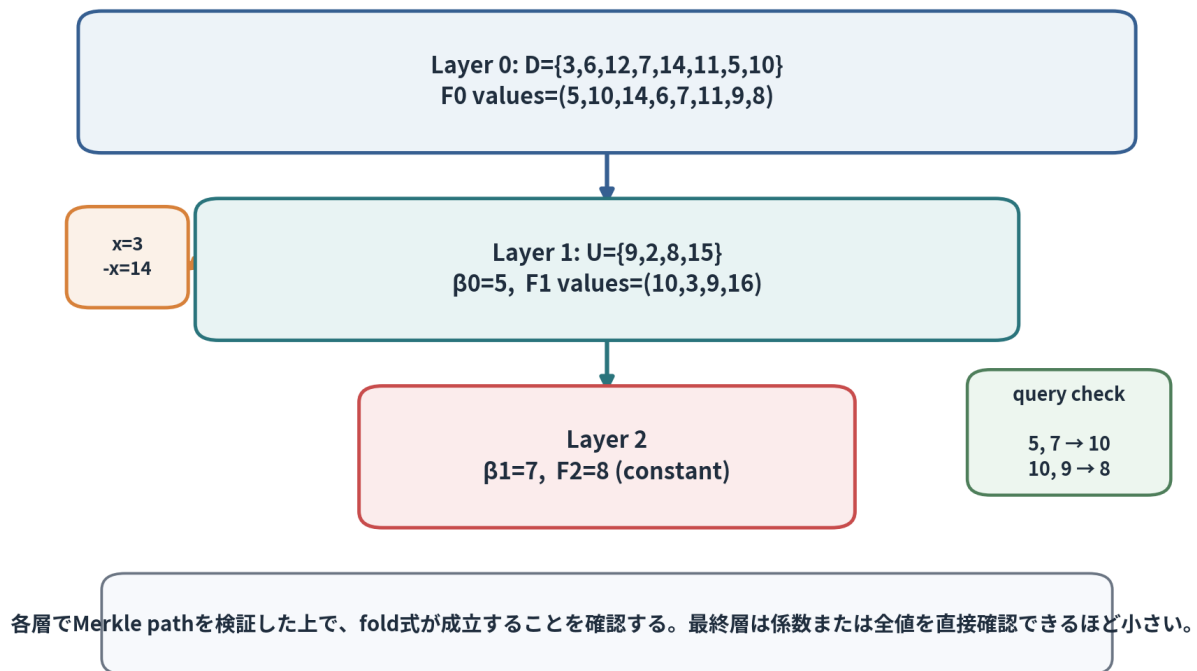


図 14 F_17 Toy Computation における FRI query path (筆者作成)

ステップ 10 検証者は、final polynomial の次数と評価を確認し、受理または拒否する

最終 FRI 多項式は十分に小さいため、検証者は係数全体を受け取り、次数上限を直接確認できる。query path の最終値が final polynomial の対応点評価と一致し、これまでのすべての検査が成功した場合のみ proof を受理する。検証は論理積であり、一つでも不一致があれば拒否する。

受理判定のフロー: 早期rejectと最終AND

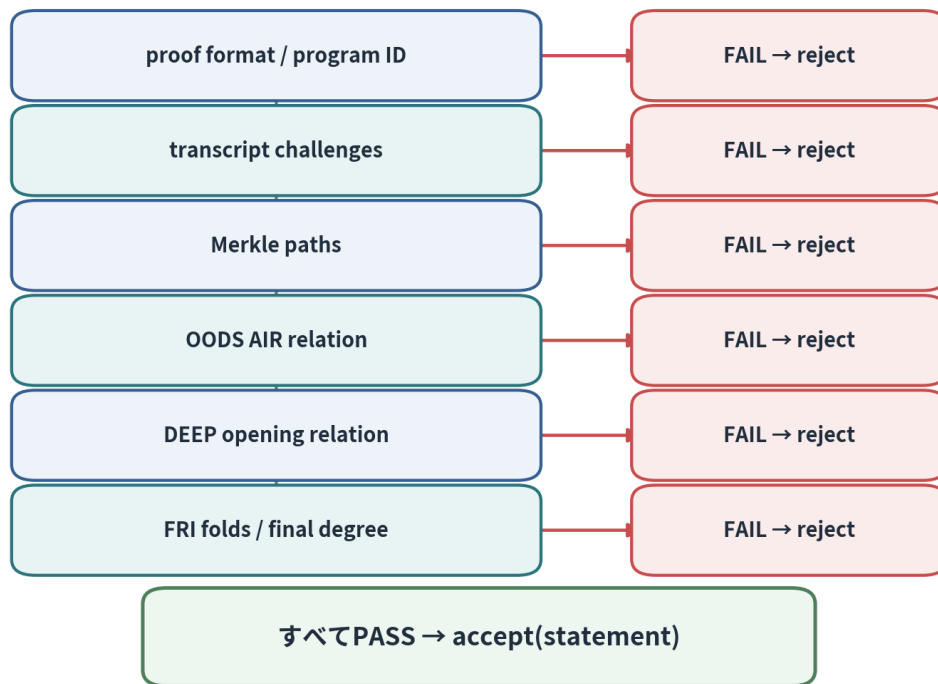


図 15 全検査の AND としての受理判定（筆者作成）

受理条件 検証者は、形式・transcript・Merkle・AIR/OODS・DEEP・FRI・最終次数のすべてを満たした場合にのみ受理する。各検査は代替関係ではなく論理積である。

第7章 攻撃と検査の対応、パラメータ、実装上の注意

7.1 どの不正をどの検査が捕捉するか

不正・実装ミス	主に検出する検査	補足
opening 値を後から差し替える	Merkle path	root より前の commit 順序も必要
別プログラムの正しい proof を提出	program/AIR ID、public input binding	verifier 側で固定する
誤った状態遷移を正しく再符号化	AIR、境界制約、ODS composition check	FRI 単独では捕捉できない
trace と composition を別々に作る	ODS/DEEP、query opening relation	複数多項式を同一主張へ接着
任意の評価表を commit する	FRI proximity test	十分遠い語を高確率で拒否
FRI 層を challenge 後に作り替える	commit-then-challenge transcript	Fiat-Shamir 順序が重要
query 数や blowup を弱く設定する	verifier policy / manifest check	proof 提示値を信用しない
同じ proof を別セッションで再利用	nonce / context を public input へ含める	アプリケーション層の binding

7.2 パラメータは互いに独立ではない

主なパラメータと検証・証明コストのトレードオフ

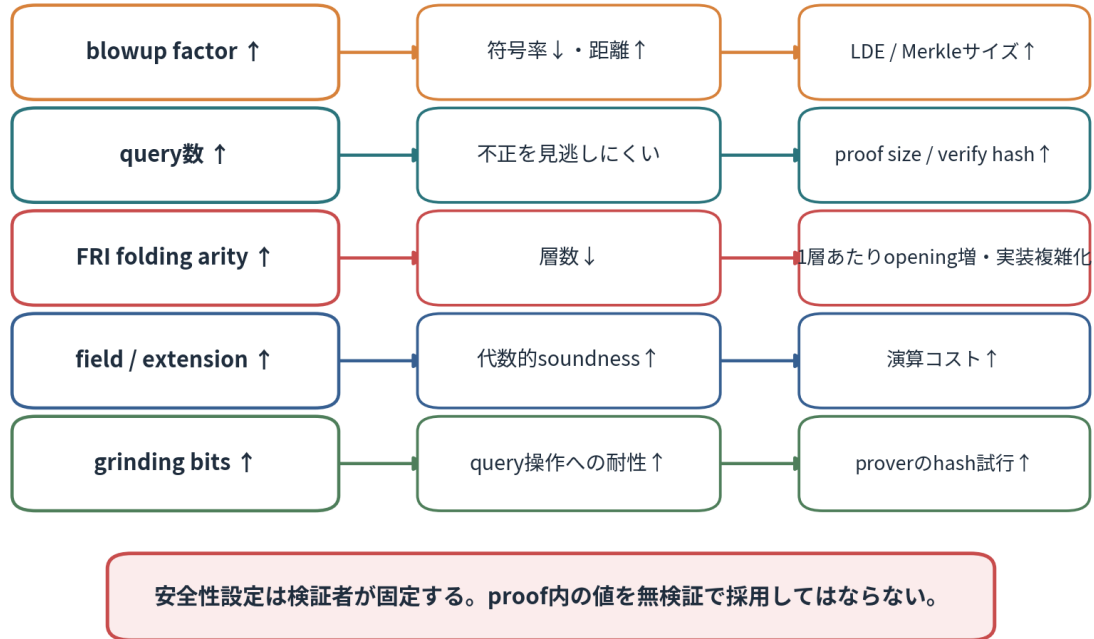


図 16 主要安全性パラメータのトレードオフ（筆者作成）

blowup factor を増やすと RS 符号率が下がり、符号語集合の相対距離は増えるが、LDE と Merkle 木が大きくなる。query 数を増やすと、遠い oracle を見逃す確率は低下するが、proof 内の opening と hash witness が増える。folding arity を上げると FRI 層数を減らせる一方、一回の fold で必要な値が増える。field size、extension degree、grinding も soundness へ影響する。最終的な security bits は、これらを合成した解析で決める必要がある。

7.3 検証コストの概観

一つの Merkle opening は、二分木なら概ね $\log_2 n$ 個の兄弟 hash を必要とする。q 個の query と L 層の FRI を単純に数えると、hash 検証はおおむね $qL \log n$ に比例して見えるが、実装では同じ path の共有、複数列の同時 commit、異なる log size、FRI arity などによって圧縮される。FRI 原論文は対話型モデルで線形証明者算術量と対数的検証者算術量を示しており、実用 STARK は Merkle commitment と Fiat-Shamir を重ねて非対話化する。[3]

7.4 実装監査で特に確認すべき事項

- public input、program ID、proof version、domain separation tag が transcript へ一意に取り込まれているか。
- field element、index、length、root、final polynomial の serialization に複数の解釈がないか。
- query 数、blowup、extension field、grinding、degree bounds を prover が弱められないか。
- OODS 点が評価領域や分母ゼロ点を避けるよう検査されているか。
- 同じ Merkle leaf へ複数列を詰める場合、列順・log size・index の binding が完全か。
- FRI layer の domain mapping、pairing $x \leftrightarrow -x$ 、fold challenge の順序が仕様どおりか。
- 最終多項式の次数上限を係数数だけでなく、不要な高次係数がゼロであることまで確認するか。

監査上の原則 安全性パラメータと検証対象の識別子は、証明者が proof 内で自由に指定する値ではなく、検証者または承認済み manifest が固定する。transcript と serialization も含めて一つの検証仕様として監査する必要がある。

Appendix A F₁₇ 上の Reed-Solomon 符号化と距離

A.1 評価領域と多項式

有限体 F₁₇ 上で、トレース領域 H={1,4,16,13}、LDE 領域 D={3,6,12,7,14,11,5,10}を用いる。次数 3 以下の多項式を次のように定める。

$$f(X)=13+X+3X^2+3X^3 \text{ over } F_{17}$$

H 上の message evaluations と D 上の codeword は次のとおりである。

点	1	4	16	13
f(h)	3	2	12	1

点	3	6	12	7	14	11	5	10
f(d)	5	10	14	6	7	11	9	8

A.2 [8,4,5] RS コード

D 上の次数 4 未満の評価コードは、n=8、k=4、最小距離 d_{min}=8-4+1=5 を持つ。最小距離が実際に達成される例として、h(X)=(X-3)(X-6)(X-12)を加えた g=f+h を考える。h は D の最初の 3 点で 0 になり、残り 5 点では非零となる。

$$h(X)=5+7X+13X^2+X^3, \quad g(X)=1+8X+16X^2+4X^3 \text{ mod } 17$$

d	f(d)	g(d)	比較
3	5	5	一致
6	10	10	一致
12	14	14	一致
7	6	3	相違
14	7	13	相違
11	11	5	相違

d	f(d)	g(d)	比較
5	9	6	相違
10	8	3	相違

確認 f と g の符号語は 3 位置で一致し、5 位置で異なる。したがって距離は 5 であり、 $d_{\min}=5$ が達成される。

A.3 トレースの 1 値を 12 から 13 へ変えた場合

H 上の message を (3,2,12,1) から (3,2,13,1) へ変えると、補間多項式は $f(X)=9+5X+16X^2+7X^3$ となる。D 上の評価は全 8 点で変化する。これは、一つの message 変更が LDE 全体へ波及する具体例である。ただし $f|_D$ も正規 RS 符号語であり、低次数性だけなら問題はない。

d	元の f(d)	変更後 f'(d)	差分 mod 17
3	5	0	12
6	10	2	9
12	14	2	5
7	6	16	10
14	7	0	10
11	11	12	1
5	9	0	8
10	8	6	15

A.4 LDE 評価の 1 シンボルだけを改ざんした場合

codeword $c=(5,10,14,6,7,11,9,8)$ の第 3 成分だけを 15 へ変えた $w=(5,10,15,6,7,11,9,8)$ は、 c から Hamming 距離 1 にある。 w は通常 $RS_D(4)$ に属さないが、符号集合には近い。FRI の proximity という語は、このような「符号語からの距離」を対象にしている。

Appendix B F₁₇ 上の FRI query 検証

B.1 初期多項式と評価値

初期多項式 $F_0(X)=13+X+3X^2+3X^3$ を D 上で評価すると、 $(5,10,14,6,7,11,9,8)$ となる。 D は前半の点と後半の負点に対応する順序 $D=(3,6,12,7,-3,-6,-12,-7)$ を取っている。

B.2 第1fold: $\beta_0=5$

$$F_1(x^2) = (F_0(x)+F_0(-x))/2 + 5 \cdot (F_0(x)-F_0(-x))/(2x)$$

query で $x=3$ を選んだ場合、 $F_0(3)=5$ 、 $F_0(-3)=F_0(14)=7$ である。 F_{17} では $2^{\{-1\}}=9$ 、 $6^{\{-1\}}=3$ であるため、次のように計算できる。

$$F_1(9) = (5+7)/2 + 5 \cdot (5-7)/6 = 6 + 4 = 10 \pmod{17}$$

第1層の多項式は $F_1(U)=1+U$ であり、 $U=\{9,2,8,15\}$ 上の評価は $(10,3,9,16)$ である。検証者は、Layer 0 の二つの値、Layer 1 の値、それぞれの Merkle path を検証した後で、この fold 等式を確認する。

B.3 第2fold: $\beta_1=7$

次の層では $U=9$ と $-U=8$ の値 10 と 9 を用いる。

$$F_2(9^2) = (10+9)/2 + 7 \cdot (10-9)/(2 \cdot 9) = 1 + 7 = 8 \pmod{17}$$

$F_1(U)=1+U$ の偶数成分と奇数成分はいずれも定数 1 であるため、 F_2 は定数 8 となる。最終層が定数であることは検証者が直接確認できる。

FRI query path: 同じランダム経路を各層で追跡する

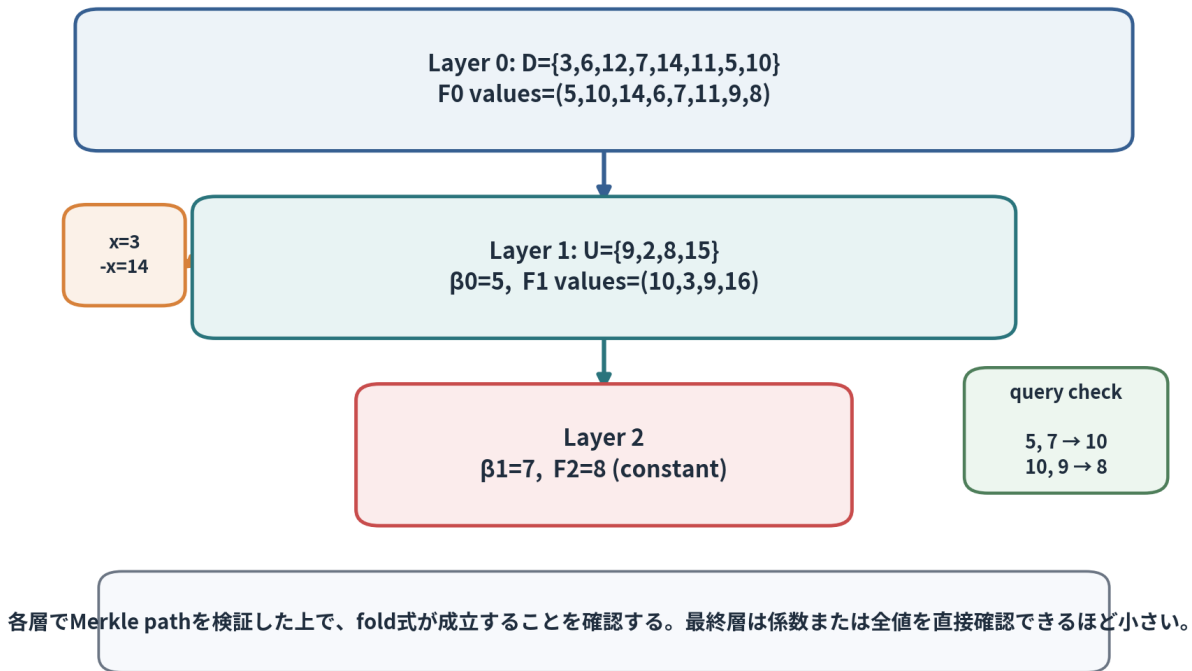


図 17 Toy Computation における二段階 FRI query の数値対応 (筆者作成)

B.4 query 検証で同時に確認しているもの

検査	使用する値	失敗時に意味すること
Merkle inclusion	各層の開示値と siblings	先に commit した oracle の値ではない
Layer 0→1 fold	$F_0(x), F_0(-x), \beta_0, F_1(x^2)$	第1層が正しく fold されていない
Layer 1→2 fold	$F_1(u), F_1(-u), \beta_1, F_2(u^2)$	第2層が正しく fold されていない
final polynomial	係数、degree bound、最終評価	最終層が主張次数に収まらない

注 3 この 8 点 Toy Computation は暗号的な soundness を持たない。実用方式では大きな有限体、より大きな評価領域、複数 query、extension field、grinding 等を組み合わせる。

まとめ Reed-Solomon 符号化と検証を一文脈で捉える

ZK-STARK における Reed-Solomon 符号化は、実行トレースを単なる表から、低次数多項式の評価という大域的に結び付いた構造へ変換する。符号率と最小距離は、評価ベクトル同士がどの程度離れるかを定め、FRI は commit された oracle がその符号集合へ近いことを少数照会で検査する。

しかし、valid RS codeword であることは、正しい計算を意味しない。誤ったトレースも一貫して再符号化できるためである。AIR と境界制約が計算の意味を定め、composition polynomial と OODS/DEEP がその意味を低次数検査へ接着し、Merkle commitment が opening の後出し変更を防ぐ。検証者は、この全連鎖を Fiat-Shamir transcript の順序どおりに再構成し、少数の query path を検査することで、計算全体を再実行せずに受理または拒否する。

最終要約 Reed-Solomon 符号化は「冗長性と距離」を作り、AIR は「正しい計算の意味」を作り、OODS/DEEP は「意味と低次数性の結び付き」を作り、FRI は「RS 符号への近さ」を検査する。検証者は、これらを commitment とランダム照会で一つずつ確認する。

参考文献

- [1] Irving S. Reed and Gustave Solomon, “Polynomial Codes over Certain Finite Fields,” *Journal of the Society for Industrial and Applied Mathematics*, Vol. 8, No. 2, 1960, pp. 300-304.
- [2] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev, “Scalable, transparent, and post-quantum secure computational integrity,” *Cryptology ePrint Archive*, Paper 2018/046, 2018.
- [3] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev, “Fast Reed-Solomon Interactive Oracle Proofs of Proximity,” *ICALP 2018, LIPIcs 107*, Article 14, DOI: 10.4230/LIPIcs.ICALP.2018.14.
- [4] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf, “DEEP-FRI: Sampling Outside the Box Improves Soundness,” *ITCS 2020, LIPIcs 151*, Article 5, DOI: 10.4230/LIPIcs.ITCS.2020.5.
- [5] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner, “Interactive Oracle Proofs,” *TCC 2016-B, LNCS 9986*, pp. 31-60.
- [6] Starknet Documentation, S-two Book, “STARK Prover,” “STARK Verifier,” “Vector Commitment Scheme,” and “Merkle Verifier,” accessed 2026-08-12.
- [7] Starknet Documentation, “SHARP - Onchain verifier,” accessed 2026-08-12.
- [8] Meta / Facebook, “Winterfell: A STARK prover and verifier for arbitrary computations,” official GitHub repository.
- [9] Google Security Research, “Critical bugs in Facebook/Polygon Winterfell library,” GHSA-8fhq-pf83-pv93, published 2023-04-12.
- [10] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf, “Proximity Gaps for Reed-Solomon Codes,” *Journal of the ACM*, Vol. 70, No. 5, 2023.

資料内の図・数値について

図はすべて本資料のために作成した概念図である。F₁₇ Toy Computation の数値は有限体上で再計算し、Reed-Solomon codeword、最小距離例、FRI folding の各等式を整合させている。実用 STARK の証明フォーマット、評価領域、folding arity、hash 関数、OODS/DEEP 構成は実装によって異なるため、本資料の図は共通構造を示す抽象化である。